

InvestigateIQ

Trace the evidence. Own the decision.

Technical flow + code walkthrough

How an alert becomes an evidence-linked draft, then a human decision.



A narrow problem, with a clear boundary

? The investigator's problem

A monitoring alert is only a signal. Someone must connect customer context, transactions, relationships, documents and guidance before reaching a reasoned conclusion.

! What this product does

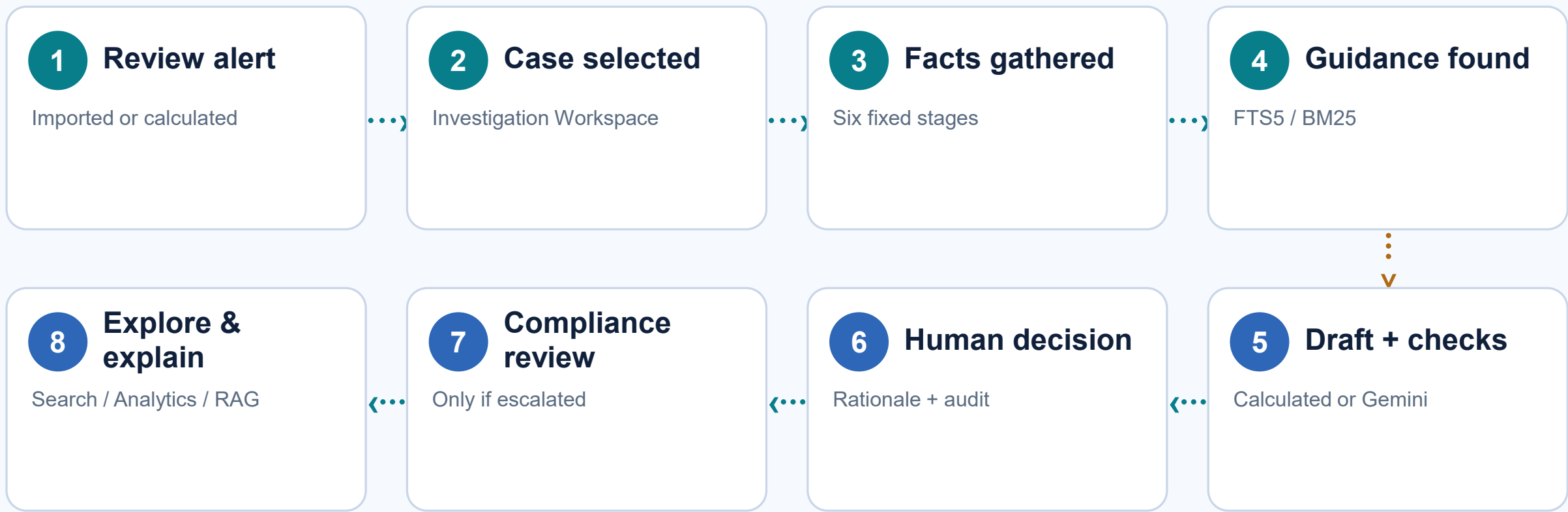
One workspace assembles those records, retrieves relevant synthetic playbook text, drafts a cited report, checks selected claims and records a human action.

BUILT: investigation workflow

NOT BUILT: real bank feed or filing

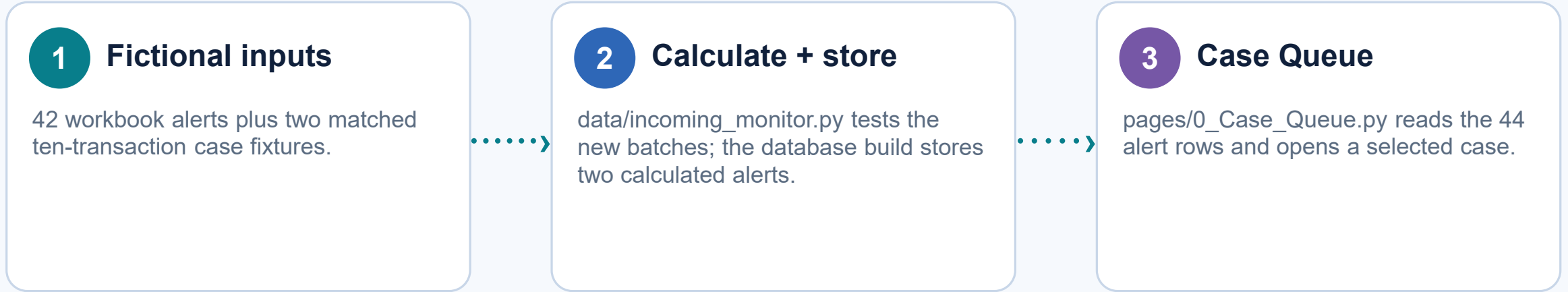
DATA: fictional

One case, end to end



Escalation is a human action. “Referred” is recorded internally; the app does not file an STR.

Where does the alert come from?



Future integration point

A bank-owned monitoring platform could send alerts through an authenticated API or batch feed into an ingestion layer. That endpoint, contract, validation and queue consumer do not exist in this code.

Rule Config only previews sensitivities; changing sliders does not regenerate existing alerts.

CASE-043: calculate the signal; inspect the gaps

Arjun Mehta · ACC-TEST-01 · fictional

4 credits ₹300,000 each = ₹1,200,000

6 debits ₹180,000 each = ₹1,080,000

Baseline ₹80,000 monthly credit (six months)

Rule result 15× baseline · 90% out · 6 beneficiaries

 Evidence still missing

No original PAN/OVD files, sender identities, independent source-of-funds proof, or external-bank posting confirmation.

Customer explanation is uncorroborated.

The same ten-transaction signal also fires for CASE-044. Neither case is a fraud verdict.

Two data stores, two jobs



Source SQLite

`data/investigateiq.db`

Customers 554 • Accounts 676
Transactions 9,982 • Alerts 44
Case source records 1,770
Case chunks 4,282 • KB chunks 57



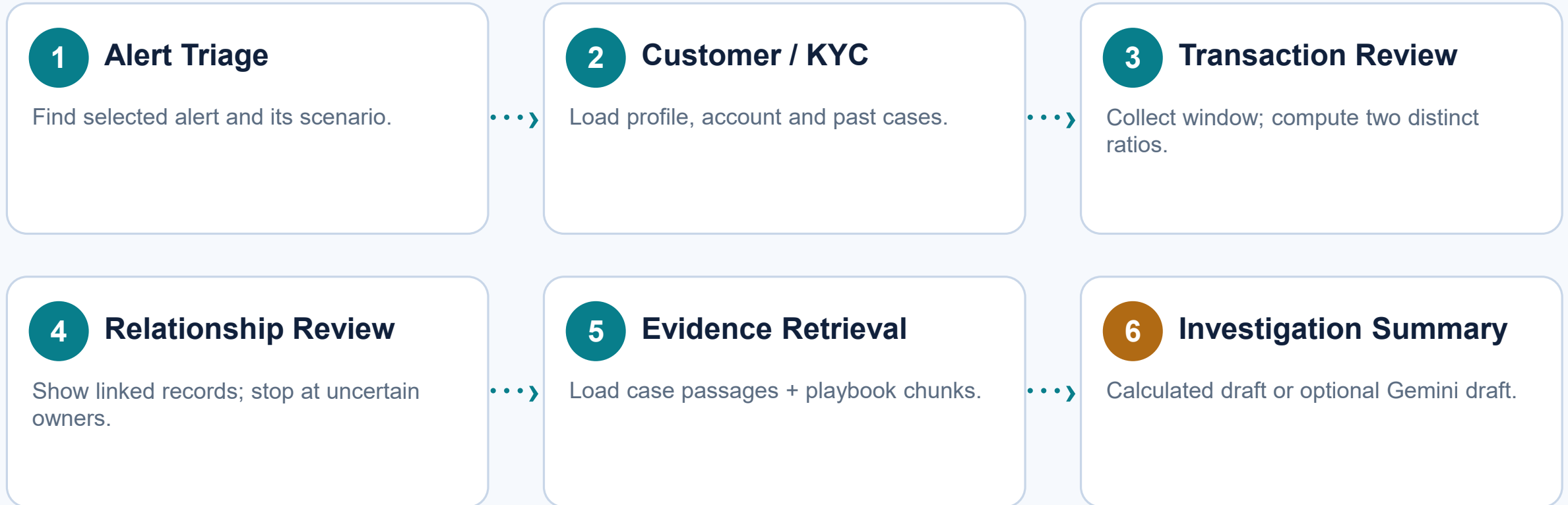
Runtime SQLite

`data/investigateiq_runtime.db`

Human actions and rationale
Audit events from each stage
Admin rule settings and change history
Mounted Railway volume preserves this state
across deployments.

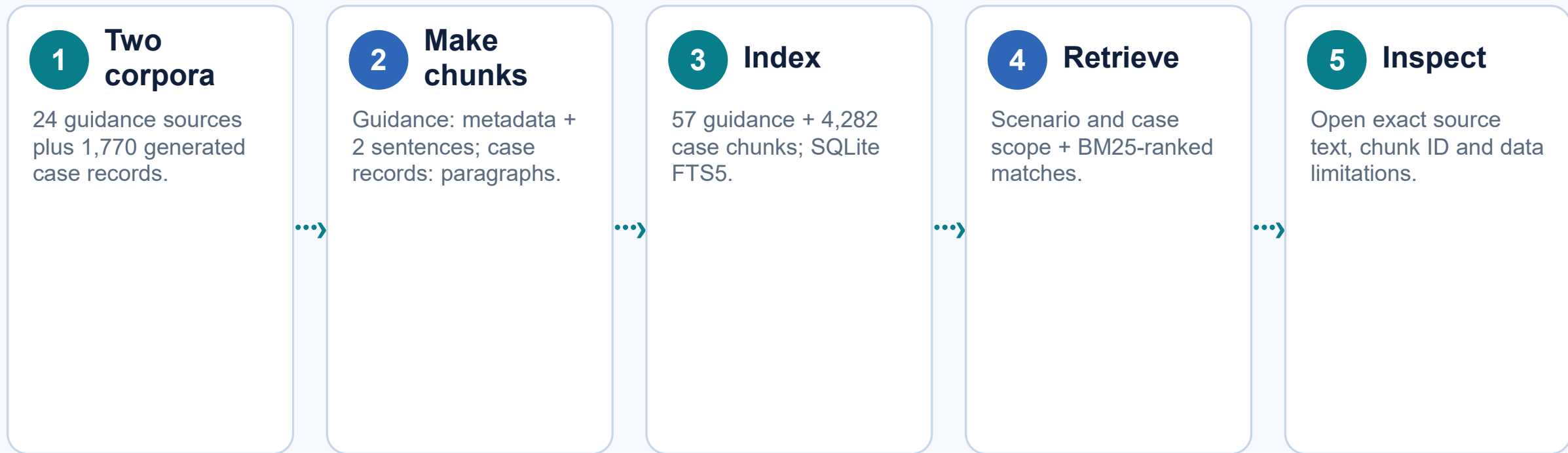
IDs link cases to rows and chunks. One source account ID has two owners: attribution is withheld.

Six narrow components, one fixed sequence



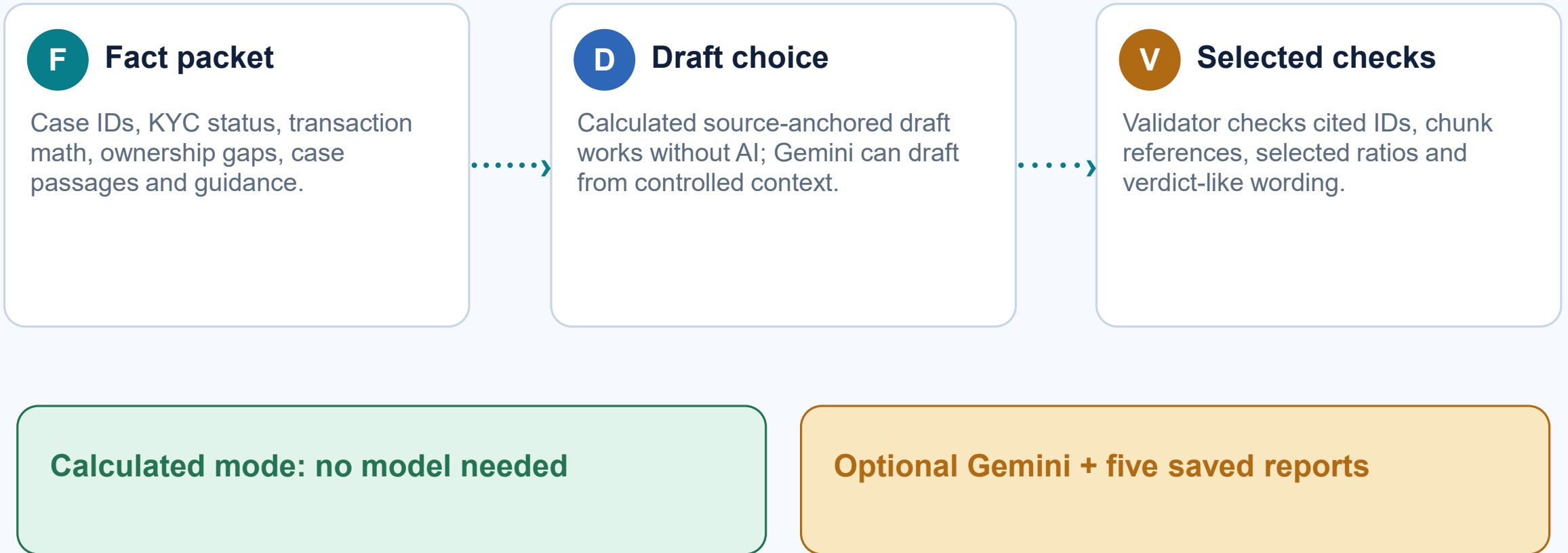
agents/orchestrator.py calls these in order and writes stage events; the validator runs after stage 6.

RAG: the answer has a visible source path



This is keyword-search RAG, not vector search. A citation points to a source; it is not proof that every sentence is correct.

From retrieved facts to a checked draft



A “PASS” is only a pass of selected automated checks; it is not a factual or legal guarantee.

The human decision is the record

C Close

Record why the alert does not need further investigation.

I Request information

Identify the missing business context or evidence.

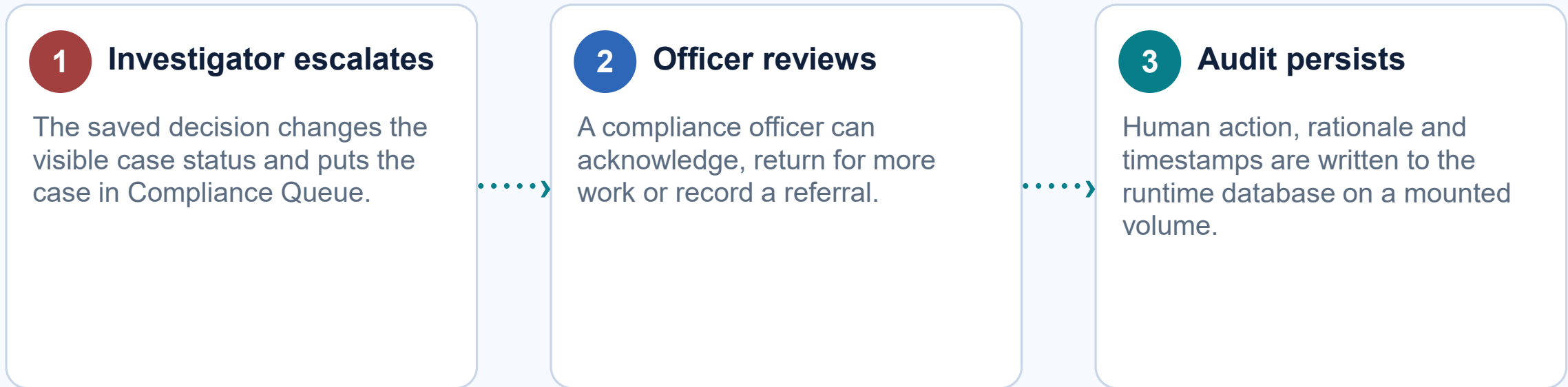
E Escalate

Send the case to a separate Compliance queue.



Investigator name + written rationale are required; accepted findings and the action are saved to runtime SQLite with an audit event.

Escalation leads to another human review



“Referred” is an internal recorded state. The app does not file an STR with FIU-IND or any regulator.

What each screen is for

1 Home

Purpose, walkthrough, entry

2 Case Queue

Filter and open alerts

3 Investigation

Evidence, chat, decision

4 Compliance

Review escalations

5 Analytics

Population trends

6 Global Search

Find records by ID/name

7 Evidence & RAG

Source/chunk inspection

8 Admin: KB

Inspect indexed guidance

9 Admin: Rules

Preview/save thresholds

Project & Team is a separate identity/roster page. Roles listed there are proposals pending team confirmation.

Why this mix of technologies?

1	Data foundation	Workbook → SQLite	Repeatable rows, IDs and calculated test alerts	data/build_database.py
2	Hybrid AI	Rules + search + Gemini	Deterministic facts before optional drafting	agents/orchestrator.py
3	RAG	Scoped chunk retrieval	Case and guidance text inspectable on-site	data/case_evidence.py
4	Responsible AI	Validator + human gate	Drafts are reviewable, not final decisions	agents/grounding_validator.py
5	Governance	Audit and Compliance	Actions have names, reasons and sequence	data/runtime_db.py

What we can show—and what remains open

✓ Verified locally

76 automated tests pass.

44 fictional alerts; two calculated from matched activity.

1,770 inspectable case records, 4,282 case chunks;
24 guidance sources and 57 guidance chunks.

→ Before real bank use

No authenticated bank feed or real identity files yet.
Resolve the ACC-1004 double-owner source error.
Obtain independent counterparty records, security review and official Indian compliance validation.

No measured fraud accuracy, time savings or ROI; no automated regulatory filing.

Proposed team responsibilities



Rahul Chainani

Application development & integration



Lakshmi

Requirements & case-study mapping



Laxman Singh

User experience & walkthrough



Pankaj

AI workflow & evidence review



RS

QA & edge-case review



Sweta Singh

Course linkage & explanation

Proposed ownership for discussion—not a record of completed coding. Confirm names and actual work before submission.

The seven code locations to remember

- | | | | |
|---|--------|--|----------------------------------|
| 1 | INPUT | <code>data/build_database.py</code> | Builds fictional source rows |
| 2 | ALERT | <code>data/incoming_monitor.py</code> | Calculates 24-hour review signal |
| 3 | FLOW | <code>agents/orchestrator.py</code> | Runs six fixed stages |
| 4 | RAG | <code>data/case_evidence.py</code> | Makes case sources and chunks |
| 5 | DRAFT | <code>agents/calculated_summary.py</code> | Builds model-free source draft |
| 6 | CHECK | <code>agents/grounding_validator.py</code> | Checks selected claims and IDs |
| 7 | DECIDE | <code>data/runtime_db.py</code> | Persists human action and audit |

“The rules flag. The evidence informs. The human decides.”